2026/11/09

OpenID TechNight vol.24 ~ SSF × AIdentity

「AIチャットからもできないの？」と言われたら

業務アプリを MCP 対応するときの認証・認可の勘所

自己紹介

土岐 孝平 （とき こうへい）

合同会社 現場指向

Java と Spring のベテランエンジニア。

開発支援・研修講師・書籍の執筆をしています。

本発表について

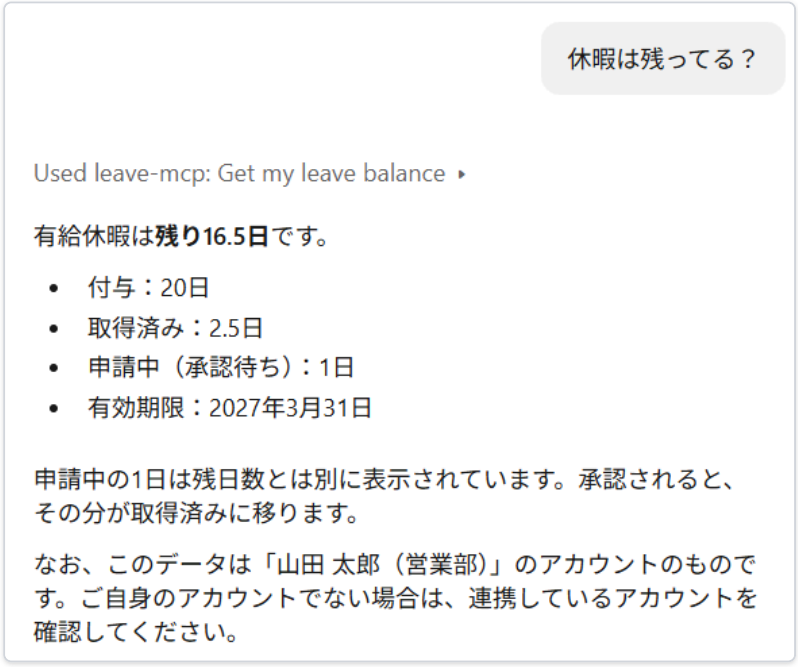
想定する聴講者	「AIチャットアプリで業務アプリの操作がしたい」と言われているシステム部門の方 業務アプリを MCP 対応する開発者の方
想定する前提知識	OAuth、OIDC、MCP についての概要を知っている
持ち帰ってもらいたいもの	MCPの認証・認可の具体的な実現イメージを掴んでもらう

やりたいこと

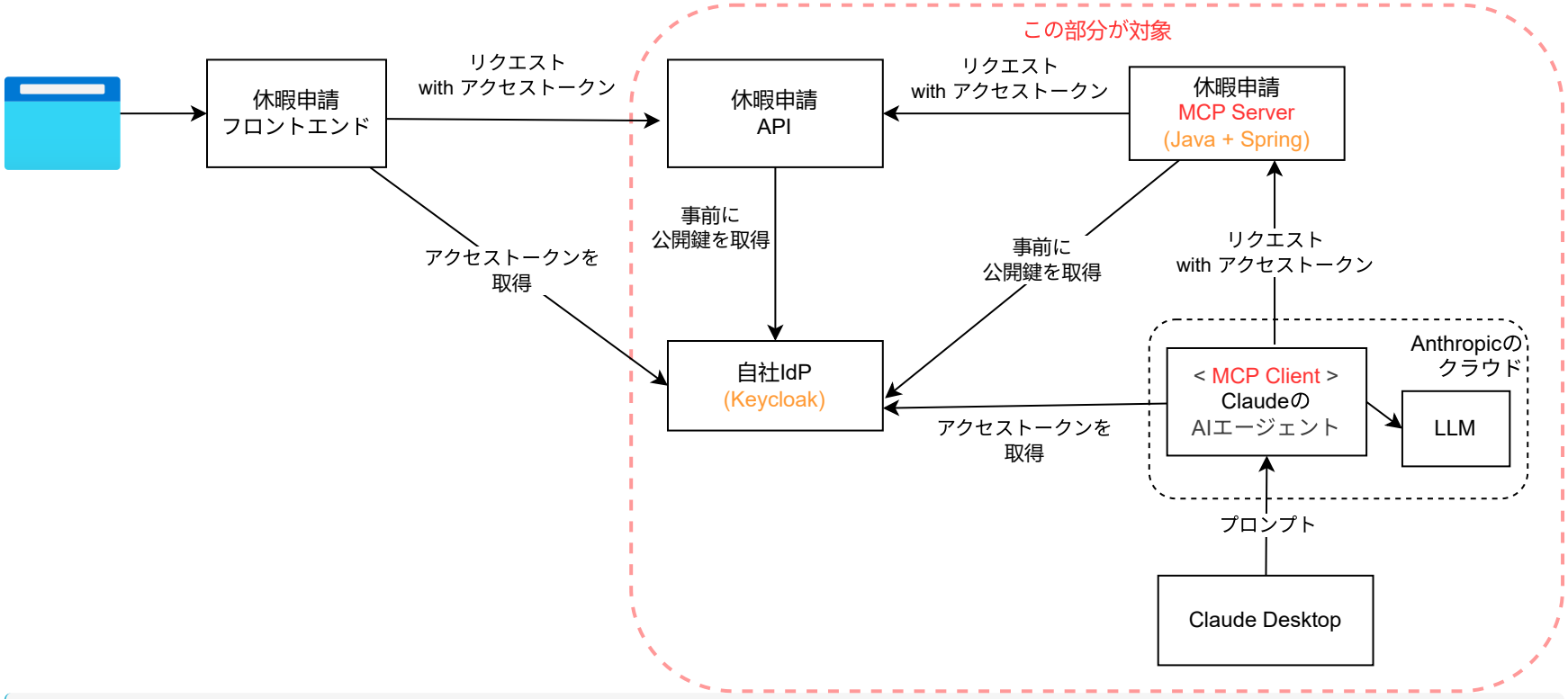
休暇申請システムをClaude Desktopで操作する





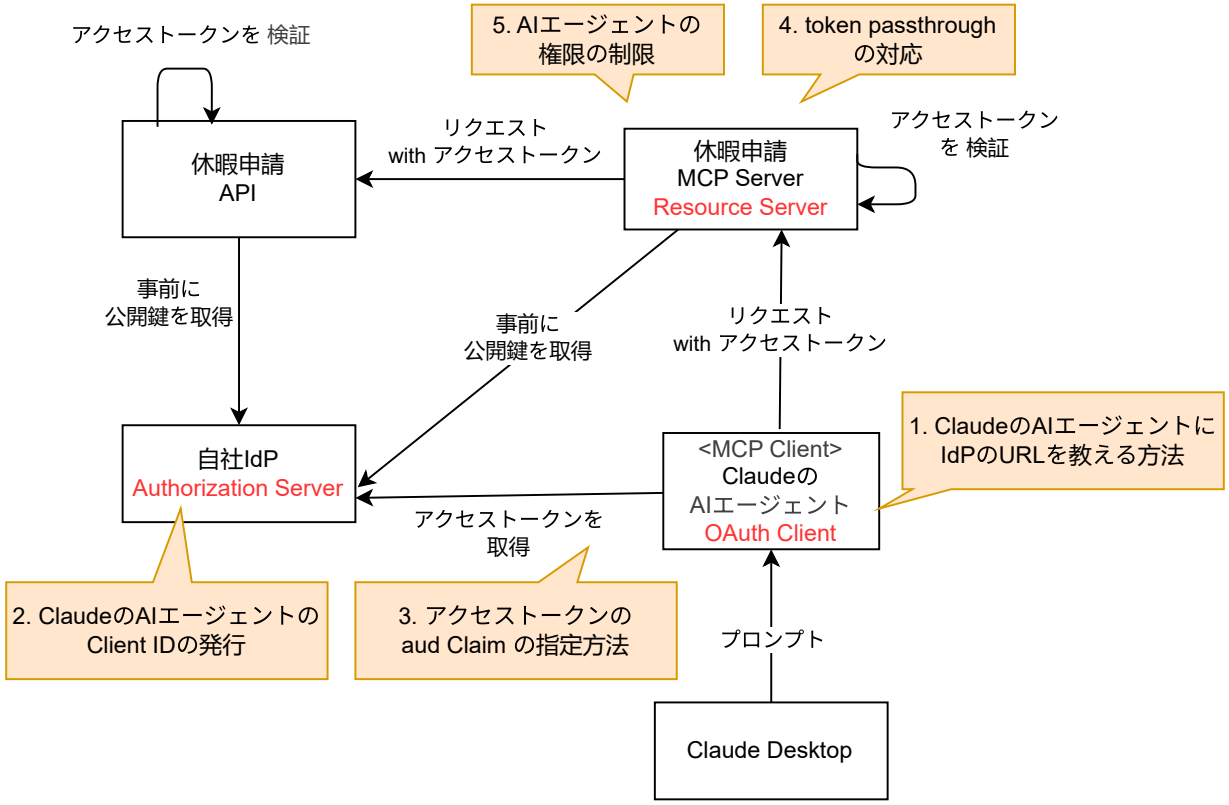


システム構成

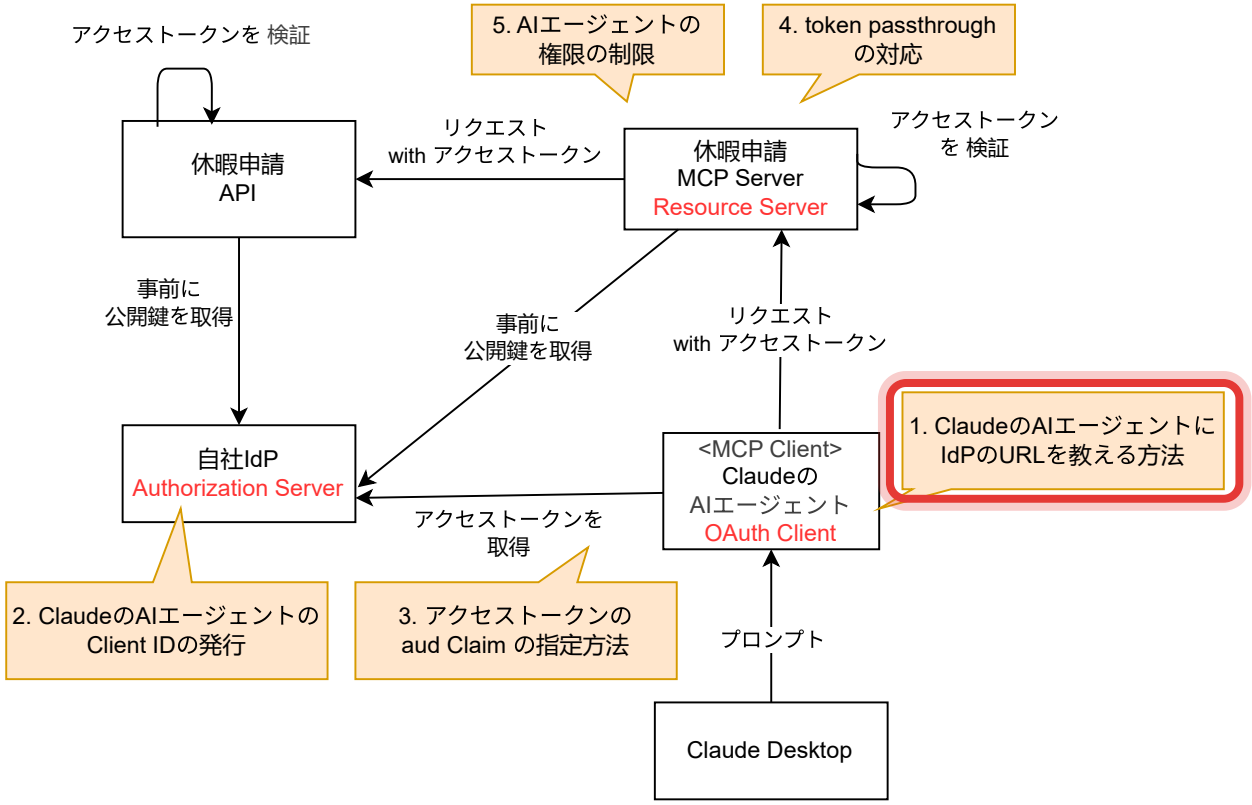


自社IdPとMCP Serverは、Claudeのサーバーからアクセスできるようにする必要がある

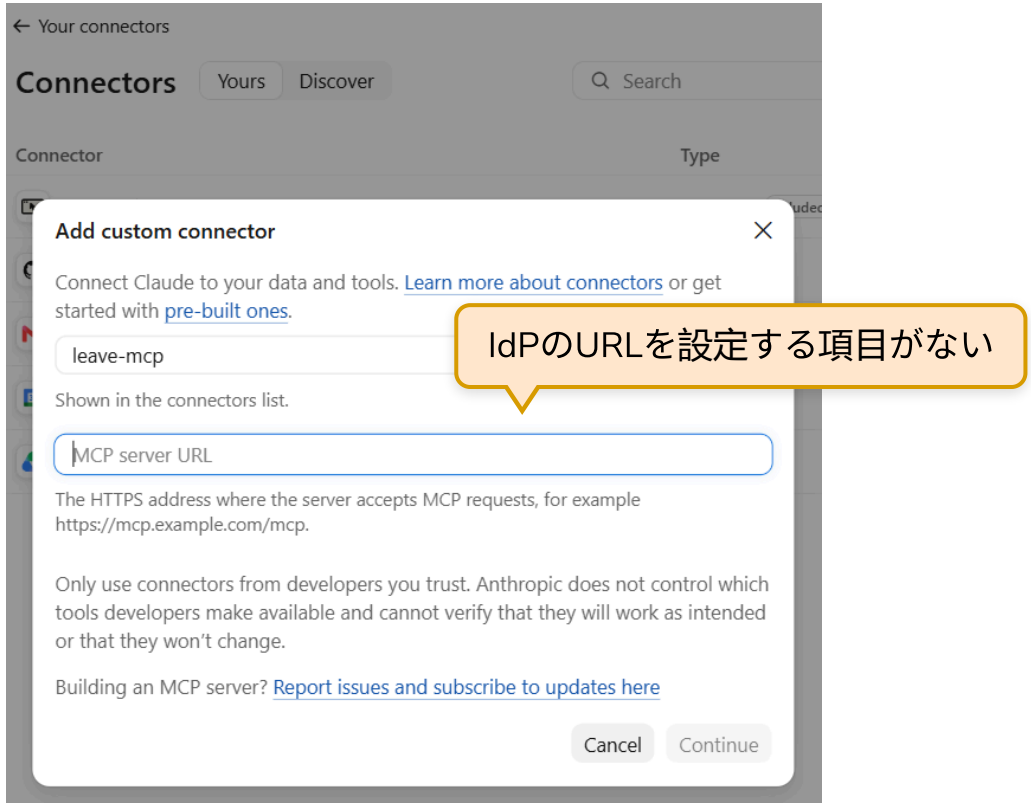
OAuthの役割と本発表の論点



1. ClaudeのAIエージェントにIdPのURLを教える方法



Claude Desktop のConnector(MCP Server)の登録画面



RFC 9728 : OAuth 2.0 Protected Resource Metadata

Resource Serverのメタデータを公開するための規格。MCP仕様では対応がMUST

① MCP Client → MCP Server

```
POST https://leave-mcp.example.com/mcp
```

② MCP Server → MCP Client

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer resource_metadata=
  "https://leave-mcp.example.com/.well-known/oauth-protected-resource"
```

③ MCP Client → MCP Server

```
GET https://leave-mcp.example.com/.well-known/oauth-protected-resource
```

④ MCP Server → MCP Client

```
{
  "resource": "https://leave-mcp.example.com/mcp",
  "bearer_methods_supported": ["header"],
  "authorization_servers": [
    "https://idp.example.com/realms/sampl-corp"
  ],
  "scopes_supported": ["leave-mcp-server"]
}
```

⑤ MCP Client → IdP（RFC 8414 : OAuth 2.0 Authorization Server Metadata）

```
GET https://idp.example.com/.well-known/oauth-authorization-server/realms/sample-corp
```

⑥ IdP → MCP Client

```
{
  "issuer": "https://idp.example.com/realms/sample-corp",
  "authorization_endpoint": "https://idp.example.com/realms/sample-corp/protocol/openid-connect/auth",
  "token_endpoint": "https://idp.example.com/realms/sample-corp/protocol/openid-connect/token",
  // ...
}
```

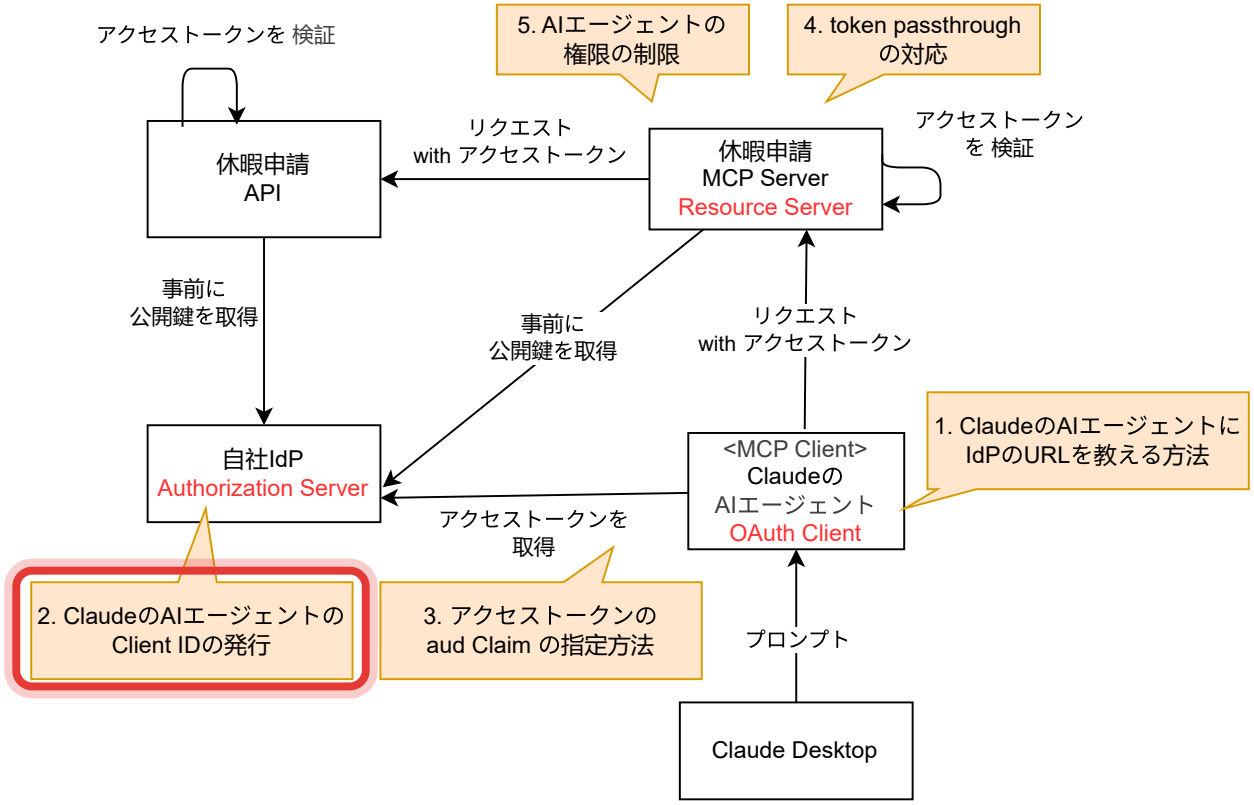
Spring を使用した実装

401レスポンス、および、メタデータのエンドポイントを自動で公開してくれる。メタデータのカスタマイズが可能

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http,
      @Value("${app.prm.resource}") String resource,
      @Value("${app.prm.scope}") String scope,
      @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuerUri) {
        ...
        http
            ...
            .oauth2ResourceServer(oauth2 → oauth2
                // authorization_servers と scopes_supported は設定が必要
                .protectedResourceMetadata(prm → prm.protectedResourceMetadataCustomizer(metadata → metadata
                    .resource(resource)
                    .authorizationServer(issuerUri)
                    .scope(scope))) // scopes_supported
                .jwt(Customizer.withDefaults()));
        return http.build();
    }
```

実際のソースコードよりも簡略化しています

2. ClaudeのAIエージェントのClient IDの発行



3つの方法

1. 事前に登録

- IdP側でOAuth Clientのメタデータ(Client ID、Client Secretなど)を発行
- それをClaudeのAIエージェントに使ってもらう

動的な割り当て

2. CIMD

OAuth Client ID Metadata Document

- 次ページで説明

3. DCR

RFC 7591 : OAuth 2.0 Dynamic Client Registration Protocol

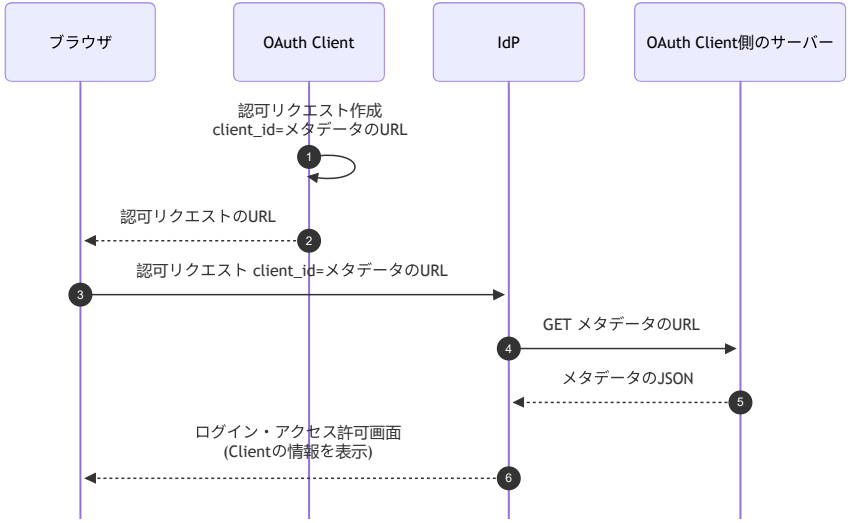
- IdPがメタデータ登録用のエンドポイントを公開
- OAuth ClientがメタデータをIdPに登録する
- メタデータの信頼性の検証が難しい

MCP仕様では非推奨

CIMDの概要

OAuth Client ID Metadata Document (IETFのドラフト)

- OAuth Client側がメタデータを公開するための企画
- 認可リクエストを作成する際に、client_idとしてメタデータのURLを設定し、認可リクエストを受信したIdPがそのURLにアクセスしてメタデータ取得する
- IdP側で信頼するドメインを設定して対象を絞ることも可能



ClaudeのCIMD: https://claude.ai/oauth/mcp-oauth-client-metadata

```
{
 "client_id":"https://claude.ai/oauth/mcp-oauth-client-metadata",
 "client_name":"Claude",
 "client_uri":"https://claude.ai",
 "redirect_uris":["https://claude.ai/api/mcp/auth_callback"],
 "grant_types":[
   "authorization_code","refresh_token",
   "urn:ietf:params:oauth:grant-type:jwt-bearer"
  ],
 "response_types":["code"],
 "token_endpoint_auth_method":"none"
}
```

Clientが選択すべき優先順位

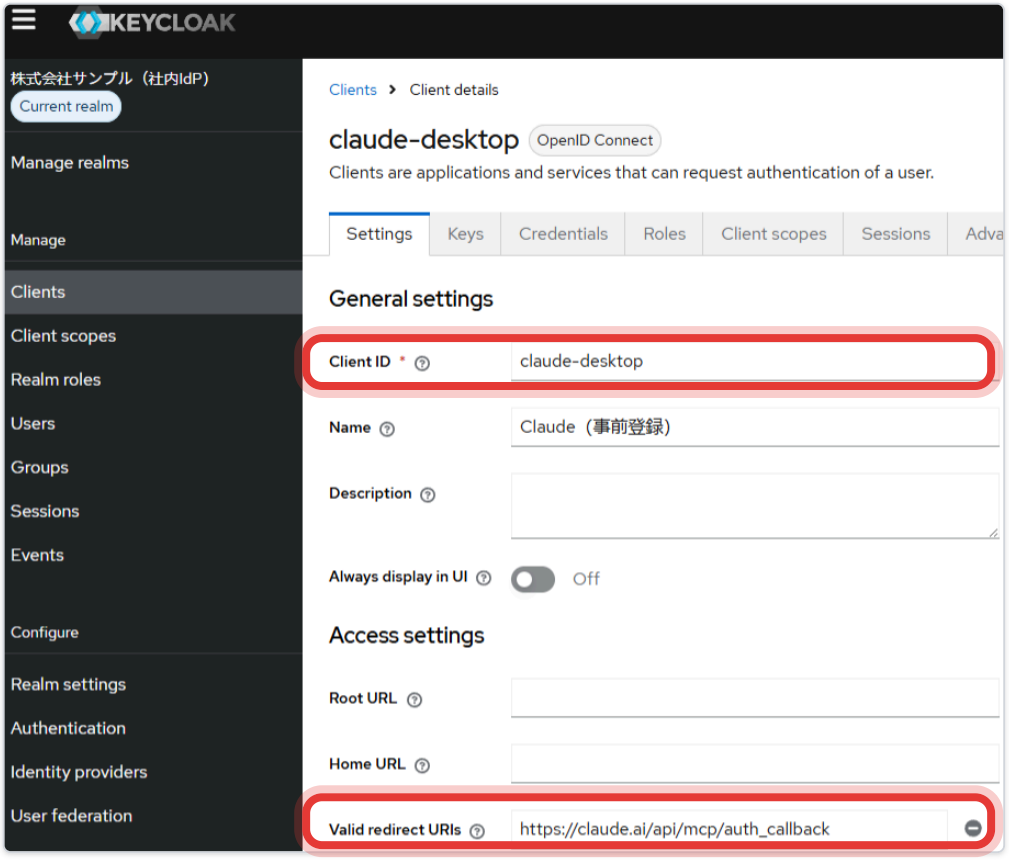
MCP仕様に記載されている、3つの選択肢が使える場合のClientが選択すべき順位

https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization/client-registration

1. pre-registered → 2. CIMD → 3. DCR

- 動的な割り当ては、未知の MCP Client が接続する場合には有効
- 業務アプリのMCP Serverの場合、MCP Clientは既知のはずなので、基本的には事前登録でよいと思われる

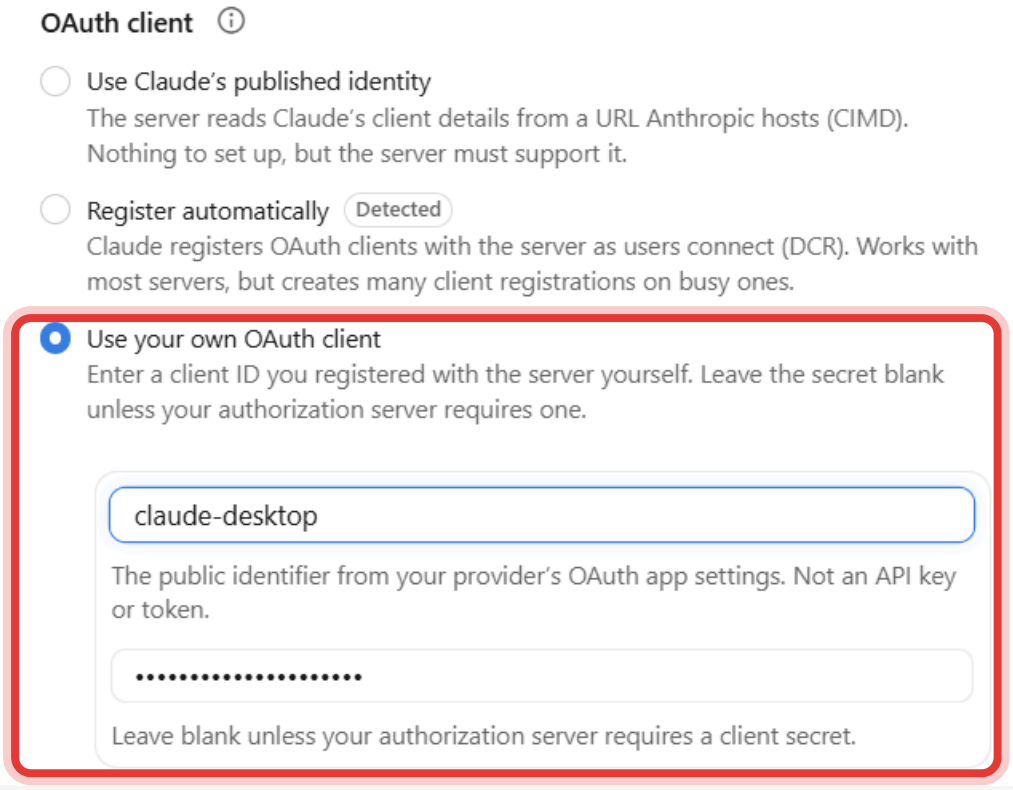
事前に登録　(keycloakの画面)



- ClaudeのAIエージェントのリダイレクトURIは、以下に記載されている

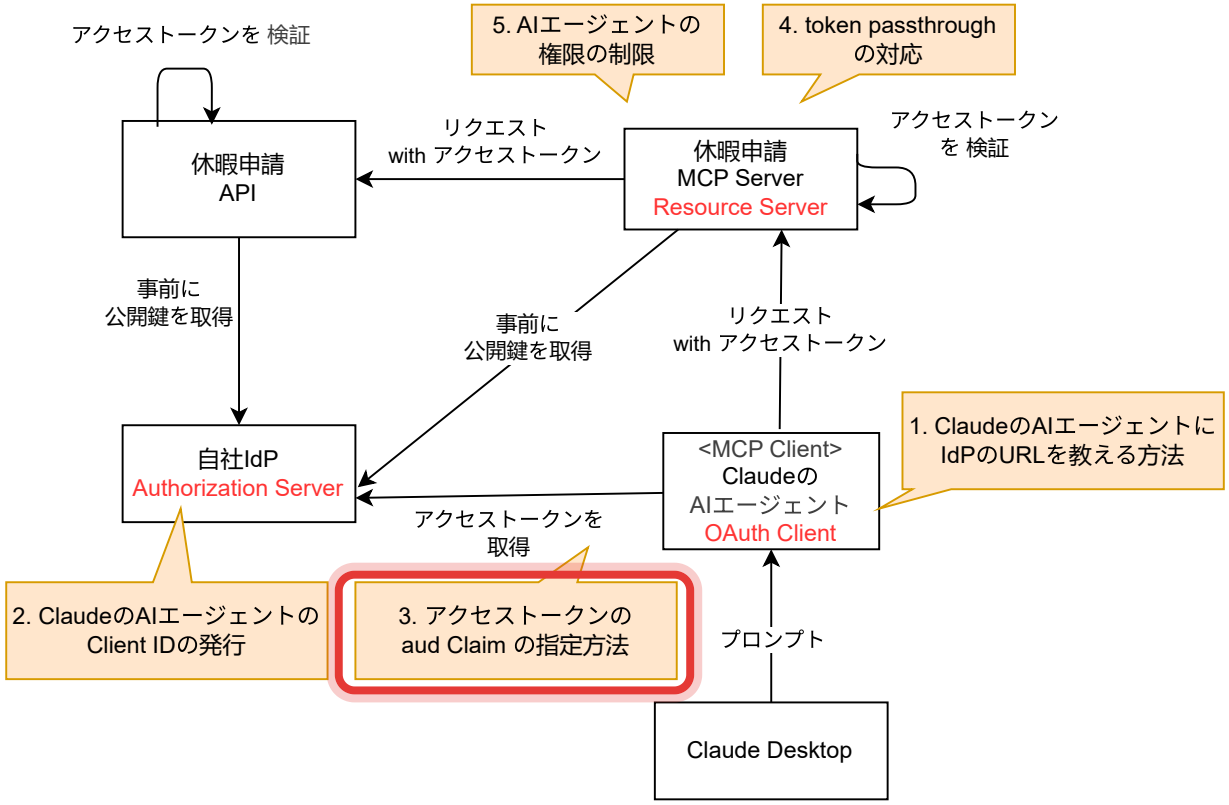
https://claude.com/docs/connectors/building/authentication#callback-urls

事前に登録　(Claude DesktopのConnector登録画面)



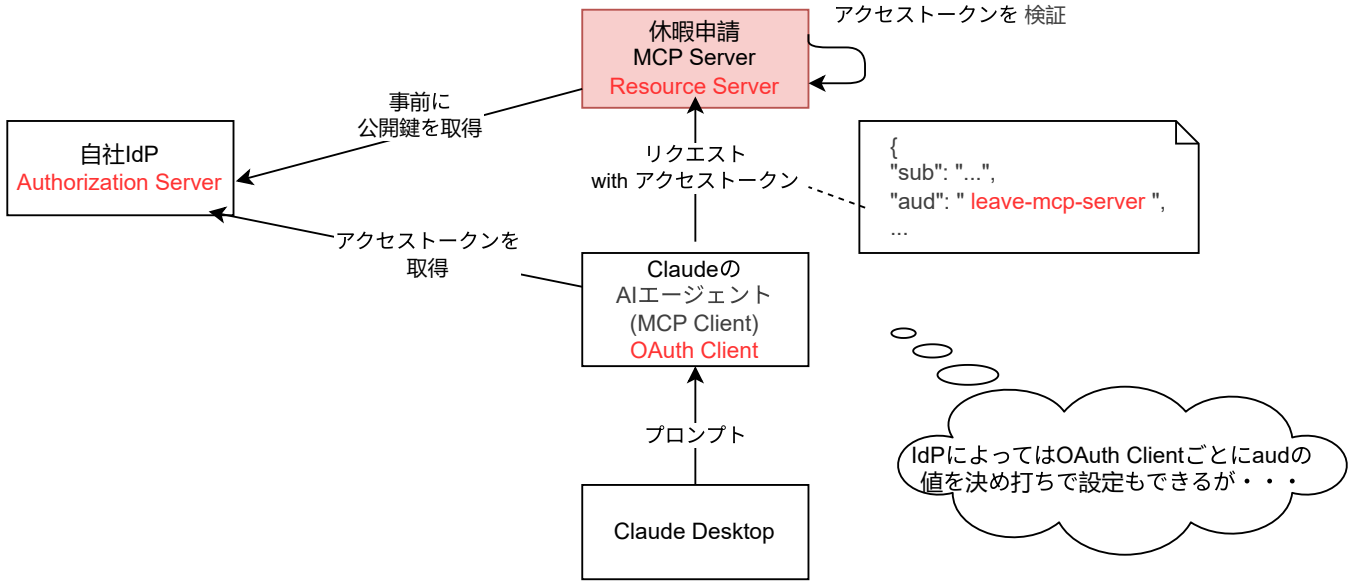
Claudeの、TeamやEnterpriseプランは、管理者がConnectorを登録し、他のメンバーは接続の操作のみにできるとのこと

3. アクセストークンの aud Claim の指定方法

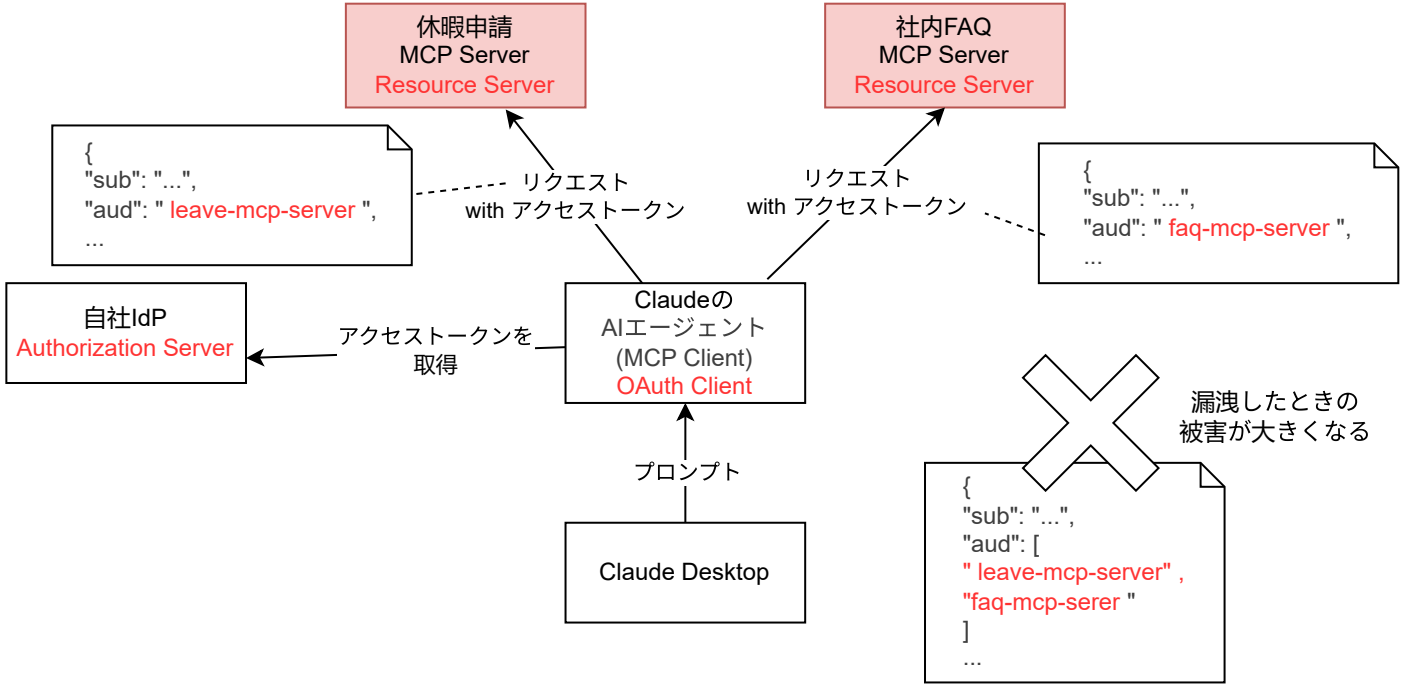


aud Claim の役割

- aud Claim は、トークンの受信者(宛先)を表す Claim
- 受信者であるResource Serverは、自分が宛先になってるかをチェックすることで、トークンが転用されるのを防ぐ
- MCP仕様では、audのチェックがMUST



MCP Serverごとに、aud の値を別にしたい



MCP Serverごとにaud Claimの値を変えるための規格

- RFC 8707 : Resource Indicators for OAuth 2.0
  - OAuth Clientが、IdPに対してResource Serverの識別子を指定するための規格

MCP Server -> MCP Client

(Protected Resource Metadata)

```
{
  "resource": "https://leave-mcp.example.com/mcp",
  "bearer_methods_supported": ["header"],
  "authorization_servers": [
    "https://idp.example.com/realms/sample-corp"
  ],
  "scopes_supported": ["leave-mcp-server"]
}
```

MCP Client -> IdP

認可リクエスト(ブラウザ経由)とトークンリクエストでresourceパラメータを指定する(RFC 8707 : Resource Indicators for OAuth 2.0)

```
https://idp.example.com/realms/sample-corp/protocol/openid-connect/auth
    ?response_type          = code
    &client_id              = claude-desktop
    &resource               = https://leave-mcp.example.com/mcp
    ...
```

IdP -> OAuth Client

発行されたアクセストークン

```
{
  "sub": "...",
  "aud": "https://leave-mcp.example.com/mcp",
  ...
```

Keycloakでの対応

- Resource Indicatorは、previewで対応している。Audience Mapper(トークンのaudに値を追加する仕組み。スコープとマッピングできる)で代用できる

MCP Server -> MCP Client

(Protected Resource Metadata)
Audience Mapperを設定したスコープを
scopes_supportedで返す

```
{
  "resource": "https://leave-mcp.example.com/mcp",
  "bearer_methods_supported": ["header"],
  "authorization_servers": [
    "https://idp.example.com/realms/sample-corp"
  ],
  "scopes_supported": ["leave-mcp-server"]
}
```

MCP Client -> IdP

認可リクエスト(ブラウザ経由)

```
https://idp.example.com/realms/sample-corp/protocol/openid-connect/auth
    ?response_type          = code
    &client_id              = claude-desktop
    &resource               = https://leave-mcp.example.com/mcp
    &scope                  = leave-mcp-server
    ...
```

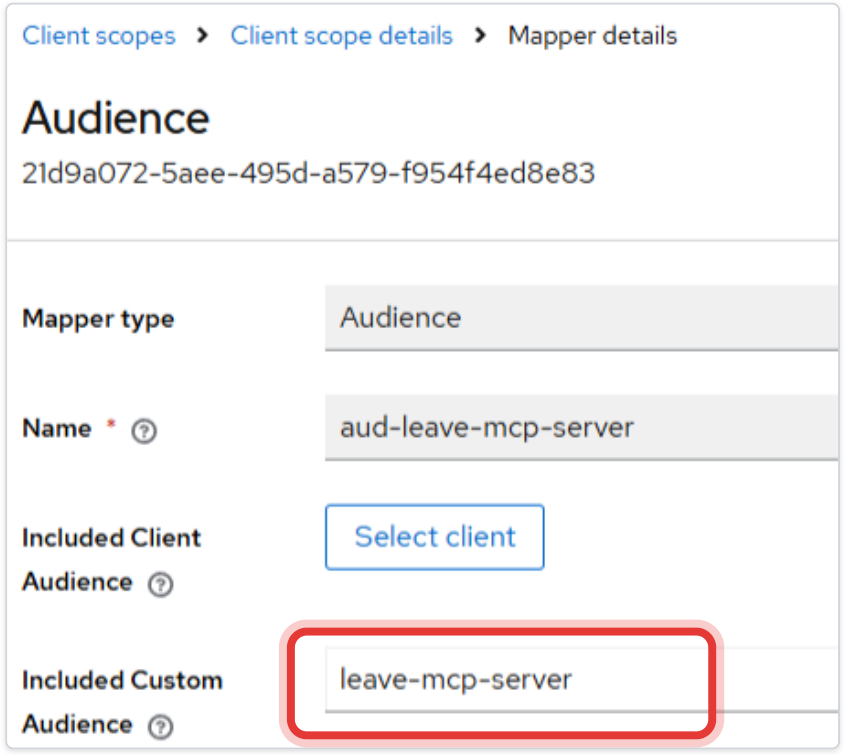
IdP -> OAuth Client

発行されたアクセストークン

```
{
  "sub": "...",
  "aud": "leave-mcp-server",
  ...
```

Audience Mapperの設定





MCP Server側(Spring)の対応

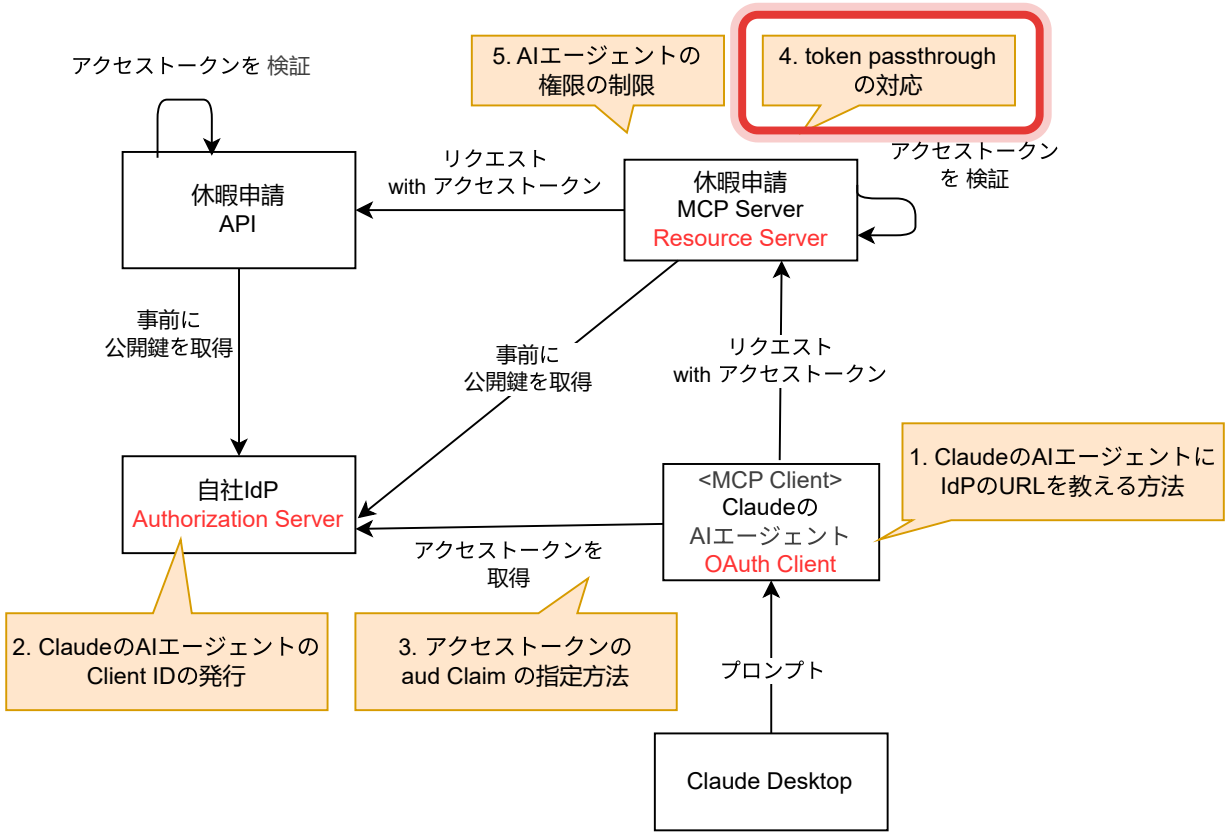
- Protected Resource Metadata の設定

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http,
  @Value("${app.prm.resource}") String resource,
  @Value("${app.prm.scope}") String scope,
   ... ) {
    ...
    http
        ...
        .oauth2ResourceServer(oauth2 → oauth2
            // authorization_servers と scopes_supported は設定が必要
            .protectedResourceMetadata(prm → prm
              .protectedResourceMetadataCustomizer(metadata → metadata
                .resource(resource)
                .authorizationServer(issuerUri)
                .scope(scope))) // scopes_supported "leave-mcp-server"を設定
            .jwt(Customizer.withDefaults()));
    return http.build();
}
```

- audの値のチェック(YAMLの設定ファイル)

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          audiences: leave-mcp-server
```

4. token passthroughの対応



token passthroughとは

token passthrough とは

MCP Serverが、受け取ったアクセストークンを、自分宛かを検証せずに、そのままAPIに渡してしまうアンチパターンのこと。MCP仕様で禁止されている

やるべき２つのこと

1. 受け取ったアクセストークンが自分宛てかを検証する(audのチェック)
2. 受け取ったアクセストークンを、API宛のアクセストークンに変換してAPIに渡す

- 1. については前述
- 2. については、RFC 8693: OAuth 2.0 Token Exchange で実現可能
  - ただし、同じ IdP 配下の API へ、アクセストークンをそのまま転送を許すかは議論されている

https://github.com/modelcontextprotocol/modelcontextprotocol/issues/3413

RFC 8693: OAuth 2.0 Token Exchange

- 手元にあるトークンを認可サーバーに渡して、別のトークンに変換してもらうための仕組み
- トークンリクエストで、grant_type「urn:ietf:params:oauth:grant-type:token-exchange」を指定

２つの方式

Impersonation方式

- 本人(subject)として操作するトークンに変換

Delegation方式

- 本人(subject)に替わって操作するトークンに変換
- Keycloakはプレビューで対応

Impersonation方式

> Keycloakの場合

リクエスト

```
POST /realms/sample-corp/protocol/openid-connect/token

grant_type              = urn:ietf:params:oauth:grant-type:token-exchange
&subject_token          = 変換前のトークン(MCP Server が受け取ったアクセストークン)
&subject_token_type     = urn:ietf:params:oauth:token-type:access_token
&client_id              = leave-mcp-server
&client_secret          = leave-mcp-server-secret
&scope                  = leave-api // Audience Mapperで紐づけたスコープ
...
```

変換前のトークン

```
{
  "sub": "340d3e2a-6d09-4d19-8a01-396d19f2d43e",
  "aud": "leave-mcp-server",
  "azp": "claude-desktop",
  ...
}
```

変換後のトークン

```
{
  "sub": "340d3e2a-6d09-4d19-8a01-396d19f2d43e",
  "aud": "leave-api",
  "azp": "leave-mcp-server",
  ...
}
```

Delegation方式

RFC 8693 の内容を転用(一部変更)

リクエスト

```
POST /as/token.oauth2 HTTP/1.1

grant_type         = urn:ietf:params:oauth:grant-type:token-exchange
audience           = urn:example:cooperation-context
subject_token      = 変換前のトークン
subject_token_type = urn:ietf:params:oauth:token-type:jwt
actor_token        = アクター(ユーザーに替わって操作する者)のトークン
actor_token_type   = urn:ietf:params:oauth:token-type:jwt
...
```

変換前のトークン

```
{
  "sub":"user@example.net",
  "aud":"https://as.example.com",
  ...
```

アクターのトークン

```
{
  "sub":"admin@example.net",
  ...
}
```

変換後のトークン

```
{
  "sub":"user@example.net",
  "aud":"urn:example:cooperation-context",
  "act": { "sub":"admin@example.net" }
  ...
}
```

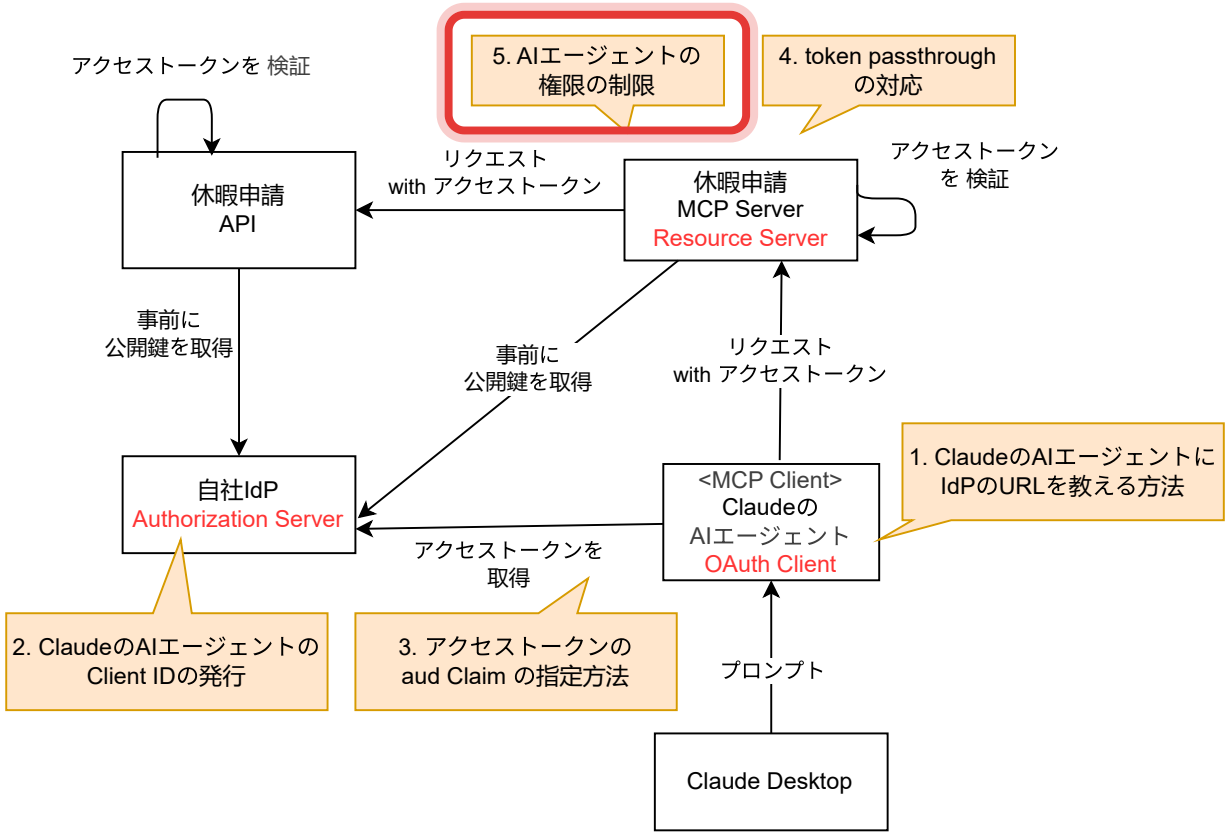
- act Claim で、アクターを表現
- sub Claim は変わらない

Token Exchangeの実装例(Impersonation方式)

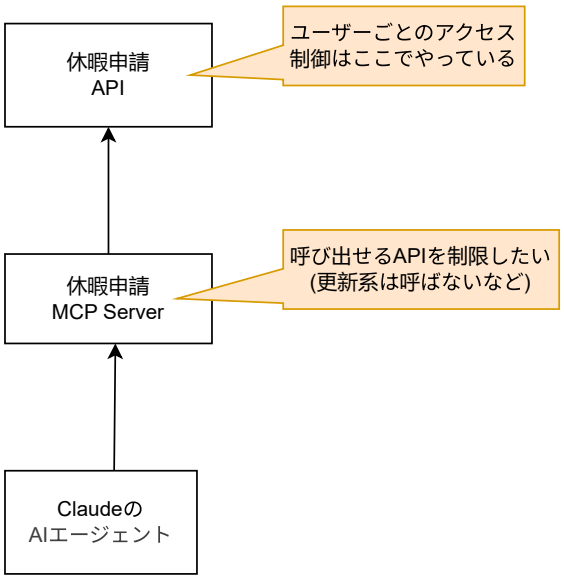
```
    @McpTool(name = "get_my_leave_balance",  annotations = @McpTool.McpAnnotations(readOnlyHint = true),
             description = "ログインしているユーザー本人のプロフィール（氏名・部署・上長・ロール）と有給休暇の残日数を取得する。")
    public JsonNode getMyLeaveBalance() {
        return restClient.get().uri("/me").header(HttpHeaders.AUTHORIZATION, "Bearer " + exchange(callerToken()))
        .retrieve().body(JsonNode.class);
    }
    private static String callerToken() {
        ((JwtAuthenticationToken)SecurityContextHolder.getContext().getAuthentication())
        .getToken().getTokenValue();
    }
    private String exchange(String subjectToken) {
        MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
        form.add("grant_type", "urn:ietf:params:oauth:grant-type:token-exchange");
        form.add("subject_token", subjectToken);
        form.add("subject_token_type", "urn:ietf:params:oauth:token-type:access_token");
        form.add("client_id", "leave-mcp-server");
        form.add("client_secret", clientSecret);
        form.add("scope", "leave-api");  // aud=leave-api を付ける Keycloak の optional scope
        JsonNode response = restClient.post().uri(tokenUri)
            .contentType(MediaType.APPLICATION_FORM_URLENCODED)
            .body(form)
            .retrieve().body(JsonNode.class);
        return response.get("access_token").asString();
    }
```

TokenExchangeOAuth2AuthorizedClientProvider を使用する方法もある

5. AIエージェントの権限の制限



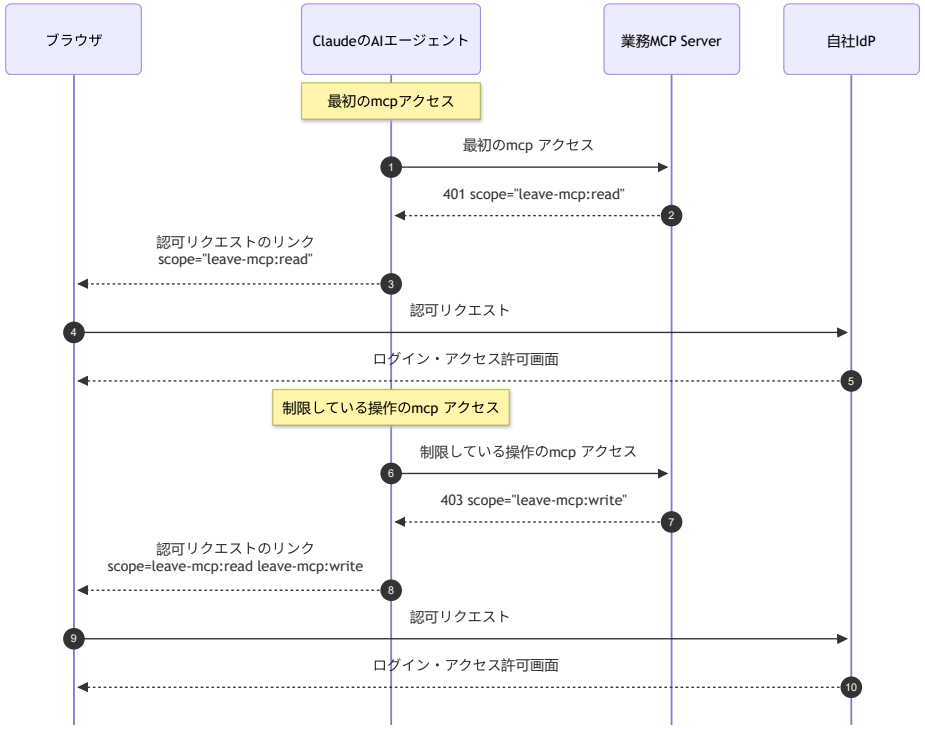
何をしたいのか



- APIでできることを、AIエージェントが勝手に操作できるのは怖いので制限したい(更新系の操作はさせたくない、など)
  - 前提として、API側はユーザーに対してアクセス制御をしているはずなので、それ以上のことをされる心配はない
  - そのうえで、AIエージェントができる範囲を制限したい
- 固定的に制限したい場合
  - 制限したいAPIを呼び出すツールを作らない
- 制限するかどうかをユーザーに判断させたい場合
  - Human In The Loop を実装
  - 同意画面でスコープを選択可能にする(Keycloakでは対応していない様子)
  - スコープを後から広げる　MCP仕様の Step-Up Authorization Flow に記載

Step-Up Authorization Flowのフロー

最初は狭いスコープでアクセストークンを発行し、必要に応じて広いスコープのアクセストークンを発行する




```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer resource_metadata=
  "https:// ... /.well-known/oauth-protected-resource"
  , scope="leave-mcp-server leave-mcp:read"
```

```
HTTP/1.1 403 Forbidden
WWW-Authenticate: Bearer error="insufficient_scope"
  , scope="leave-mcp:write"
```

RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage で示されている

```
GET /realms/sample-corp/protocol/openid-connect/auth
    ?response_type = code
    &client_id     = claude-desktop
    &scope         = leave-mcp-server leave-mcp:read
leave-mcp:write offline_access
    ...
```

```
{
    "sub": " ... ",
    "scope": "leave-mcp:read leave-mcp:write  ... ",
    ...
}
```

スコープによるアクセス制御

```
    @McpTool(name = "list_my_leave_requests",
             description = "ユーザー本人の休暇申請の一覧を新しい順に取得する。状態は PENDING(申請中), APPROVED(承認済), REJECTED(却下), CANCELED(取消)。",
             annotations = @McpTool.McpAnnotations(readOnlyHint = true))
    @McpRequiredOauthScopes("leave-mcp:read")
    public JsonNode listMyLeaveRequests() {
        return leaveApiClient.myRequests(callerToken());
    }

    @McpTool(name = "submit_leave_request",
             description = """
              ユーザー本人の休暇を申請する。申請は上長の承認待ち(PENDING)になる。
              残日数不足・過去日・同じ日の重複は休暇申請API がエラーで返す。""",
             annotations = @McpTool.McpAnnotations(destructiveHint = false))
    @McpRequiredOauthScopes("leave-mcp:write")
    public JsonNode submitLeaveRequest(
            @McpToolParam(description = "休暇の開始日 (yyyy-MM-dd) ") LocalDate startDate,
            @McpToolParam(description = "休暇の終了日 (yyyy-MM-dd) 。1日だけなら省略するか開始日と同じ", required = false) LocalDate endDate,
            @McpToolParam(description = "種別。PAID(有給, 既定), SPECIAL(特別休暇), ABSENCE(欠勤)", required = false) String type,
             ...
```

- @McpRequiredOauthScopesアノテーションは、Spring提供のものではなく自作したもの
- ツールにアクセスされた際にアクセストークンのスコープを確認し、スコープが不足していれば例外を発生させる
  https://github.com/spring-projects/spring-ai/issues/6740#issuecomment-5217399970 を参考

401 と 403のレスポンスのカスタマイズ

```java
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http,
             ...
            .oauth2ResourceServer(oauth2 -> oauth2
                .protectedResourceMetadata(prm -> prm
                  ...
                .authenticationEntryPoint(prmAwareEntryPoint(resource, scope))
                .jwt(Customizer.withDefaults()));
        return http.build();
    }

    private static AuthenticationEntryPoint prmAwareEntryPoint(String resource, String scope) {
       ...
        return (request, response, authException) -> {
            if (authException instanceof OAuth2AuthenticationException e
                    && e.getError() instanceof BearerTokenError error
                    && BearerTokenErrorCodes.INSUFFICIENT_SCOPE.equals(error.getErrorCode())) {
                // スコープが満たされてなかった場合
                response.setHeader("WWW-Authenticate",
                    "Bearer error=\"insufficient_scope\", error_description=\"" + error.getDescription()
                        + "\", scope=\"" + error.getScope() + "\", resource_metadata=\"" + metadataUrl + "\""); // "leave-mcp:write" を指定
                response.sendError(error.getHttpStatus().value(), "Forbidden");
                return;
            }
            // 単に認証されてなかった場合
            response.setHeader("WWW-Authenticate",
                "Bearer resource_metadata=\"" + metadataUrl + "\", scope=\"" + scope + "\""); // "leave-mcp:read" を指定
            response.sendError(401, "Unauthorized");
        };
    }
```

まとめ

1. ClaudeのAIエージェントにIdPのURLを教える方法	RFC 9728: OAuth 2.0 Protected Resource Metadata
2. ClaudeのAIエージェントのClient IDの発行	事前登録、CIMD、DCRが選択肢 MCP Clientが既知の場合は事前登録が無難
3. アクセストークンの aud Claim の指定方法	RFC 8707: Resource Indicators for OAuth 2.0 Keycloakの場合はScopeで代替
4. token passthroughの対応	MCP Serverでaudの値をチェック APIを呼び出す際、API宛の別のトークンに交換 RFC 8693: OAuth 2.0 Token Exchange
5. AIエージェントの権限の制限	制限するかをユーザーに委ねる場合は、Step-Up Authorization Flowが選択肢

ソースコードはこちらで公開: https://github.com/genba-oriented/mcp-oauth-sample

ユーザー名「genba-oriented」